



UNIVERSIDADE DE BRASÍLIA – UnB

Faculdade de Ciências e Tecnologias em Engenharia – FCTE

Arquitetura e Desenho de Software - FGA0208

RELATÓRIO DE CASOS DE TESTES NO SITE DECATHLON

Fluxo: Carrinho De Compras >> Checkout >> Pagamento

SubEquipe 03:

Camile Barbosa Gonzaga de Oliveira - 251035022

Letícia de Carvalho dos Santos - 222022135

Rafaela Andrea Radamés Guerra – 231031723

BRASÍLIA

2026

LISTA DE FIGURAS

<i>Figura 1: Seleção de Produto e Especificações</i>	10
<i>Figura 2: Adição do Produto ao Carrinho</i>	11
<i>Figura 3: Visualização na página do carrinho de compras.</i>	12
<i>Figura 4: Alteração de quantidade de produtos no carrinho.</i>	13
<i>Figura 5: Remoção de produtos do carrinho.</i>	14
<i>Figura 6: Página de transição para o checkout</i>	15
<i>Figura 7: Identificação do usuário e Carregamento de Perfil. Fonte: Autoria própria</i>	16
<i>Figura 8: Preenchimento de dados cadastrais e coleta de comportamento (antifraude). Fonte: Autoria própria.</i>	17
<i>Figura 9: Consulta Autocompletar de CEP</i>	18
<i>Figura 10: Seleção de opções de entrega</i>	19
<i>Figura 11: Modal de seleção de lojas físicas para retirada.</i>	20
<i>Figura 12: Seleção de método de pagamento.</i>	22
<i>Figura 13: Seleção do método de pagamento: cartão de crédito. Fonte: Autoria própria.</i>	23
<i>Figura 14: Expansão do formulário de pagamento com cartão de crédito, integração antifraude e atualização do estado do checkout. Fonte: Autoria própria</i>	24
<i>Figura 15: Consulta dinâmica de parcelamento por BIN do cartão de crédito e sincronização do orderForm. Fonte: Autoria própria</i>	26
<i>Figura 16: Seleção da modalidade PIX, atualização assíncrona do contexto de pagamento e prontidão para fechamento do pedido. Fonte: Autoria própria.</i>	27
<i>Figura 17: Validação de regra de negócio, tratamento de exceção em cupom promocional inválido e execução da limpeza de contexto. Fonte: Autoria própria.</i>	28

SUMÁRIO

1. INTRODUÇÃO	5
2. METODOLOGIA DE ENGENHARIA REVERSA E TESTES	6
2.1. Etapas Operacionais.....	6
2.2. Checklist de Inspeção.....	7
3. RESULTADOS OBTIDOS	10
Evidência 1: Seleção de Produto e Especificações (Modal/Pré-Carrinho).....	10
Evidência 2: Adição do Produto ao Carrinho e Recomendação Inteligente (IA)	11
Evidência 3: Visualização do carrinho completo; Cálculo de frete e Recomendações de página	12
Evidência 4: Alteração de Quantidade de Produtos e Recálculo Dinâmico	13
Evidência 5: Remoção de Produto e Estado de Carrinho Vazio.....	14
Evidência 6: Transição para o Checkout (Identificação / Compra rápida)	15
Evidência 7: Identificação do Usuário e Carregamento de Perfil	16
Evidência 8: Preenchimento de Dados Cadastrais e Coleta de Comportamento (AntiFraude).....	17
Evidência 10: Consulta Autocompletar de CEP (ViaCEP / Logística VTEX)	18
Evidência 11: Seleção de Opções de Entrega e Cálculo de Frete (<i>shippingData</i>)	19
Evidência 12: Modal de Seleção de Lojas Físicas para Retirada	20
Evidência 13: Seleção do Método do Pagamento e Inicialização do Checkout Final.....	21
Evidência 14: Seleção do Método de Pagamento via Cartão de Crédito e Telemetria Antifraude (<i>paymentData</i>)	23
Evidência 15: Seleção e Expansão do Formulário de Cartão de Crédito	24
Evidência 16: Consulta do BIN do Cartão e Seleção de Parcelamento	25
Evidência 17: Alternância para Pagamento via PIX e Sincronização de Estado	26
Evidência 18: Validação de Regra de Negócio – Tentativa de Aplicação de Cupom Inválido	28
4. ANÁLISE CRÍTICA DOS RESULTADOS	30
4.1. Navegação, Seleção de Produtos e Motor de Recomendação	30
4.2. Gestão do Carrinho de Compras, Logística e Reatividade de Estado	30

4.3. Transição para o Checkout, Identificação e Segurança Antifraude	31
4.4. Logística de Entrega, Autocompletar e Retirada em Loja Física (Pickup Points)...	32
4.5. Processamento de Pagamento, Antifraude e Validação de Regras de Negócio	32
5. CONSIDERAÇÕES FINAIS	34

1. INTRODUÇÃO

Este relatório tem como objetivo registrar e analisar os resultados obtidos por meio de ensaios de teste operacionais e engenharia reversa comportamental na plataforma web da Decathlon. O foco central da análise concentrou-se no Fluxo C: “Carrinho de Compras >> Checkout >> Pagamento”, abordando também o papel dos motores de Inteligência Artificial como apoio ao processo.

A engenharia reversa comportamental foi adotada como estratégia para mapear o funcionamento real do sistema a partir da observação direta de suas respostas a diferentes estímulos de entrada. Entre os estímulos aplicados, destacam-se a seleção e alteração de quantidade de produtos, a simulação de frete, a aplicação de cupons, o preenchimento de dados de entrega e o acionamento das etapas de pagamento e validações de risco.

Este levantamento empírico foi fundamental para o desenvolvimento do módulo Modelagem do projeto da disciplina de Arquitetura e Desenho de Software, pois ofereceu os insumos e evidências necessárias para a produção de artefatos subsequentes deste. Os dados e evidências coletados permitiram mapear os limites do sistema, identificar integrações com serviços terceiros e fundamentar as decisões de projeto de arquitetura tomadas pela equipe.

2. METODOLOGIA DE ENGENHARIA REVERSA E TESTES

A abordagem metodológica adotada para a execução deste estudo fundamentou-se na Engenharia Reversa combinada com o Teste de Caixa-Preta e Inspeção de Tráfego de Rede com a ferramenta DevTools (F12).

O Teste Caixa-Preta não se preocupa com a estrutura interna do programa, mas em encontrar circunstâncias nas quais a aplicação se comporta ou não de acordo com suas especificações (MYERS *et al.*, 2012).

Esta estratégia permitiu inferir as regras de negócio e os componentes arquiteturais da Decathlon sem acesso prévio ao código-fonte ou banco de dados.

2.1. Etapas Operacionais

A execução seguiu as seguintes etapas operacionais:

- **Mapeamento de Interfaces e Fluxos Críticos:** Identificação do caminho de navegação do usuário no Fluxo C, cobrindo a gestão de itens no carrinho, o cálculo logístico de frete, o preenchimento dos dados de entrega no checkout e a seleção de métodos de pagamento (Cartão de Crédito e PIX).
- **Inspeção de Requisições de Rede (DevTools / F12):** Monitoramento contínuo das chamadas de API (HTTP GET/POST/PUT) no console de rede do navegador para mapear os endpoints do back-end, payloads de requisição/resposta e o uso de plataformas subjacentes (como os serviços da plataforma VTEX).
- **Mapeamento da Atuação de Inteligência Artificial:** Identificação das chamadas de API responsáveis por disparar motores de recomendação de produtos (vitrines de cross-selling no carrinho) e scripts de segurança/antifraude (*device fingerprinting*) executados antes e durante a fase de pagamento.
- **Estímulo de Entradas e Análise de Casos de Borda:** Submissão de dados válidos e sintéticos (como alteração rápida de quantidades, CEPs inválidos e cartões com dados incorretos) para observar mensagens de erro, validações e limites operacionais da interface.

2.2. Checklist de Inspeção

A fim de mapear o máximo de regras de negócio do fluxo previamente determinado foi criado um checklist de inspeção para orientar o desenvolvimento dos testes, de forma que explorem comportamentos de borda, validações implícitas e integrações de back-end/IA.

É importante observar que o checklist é não-exaustivo, significando que outras observações podem ser extraídas das análises e foi desenvolvido de forma que a sua execução completa seja feita ao longo do tempo, isto é, pontuando funcionalidades que serão vistas no presente e outras que poderão ser checadas posteriormente sem prejuízo do resultado parcial.

Checklist de Inspeção e Engenharia Reversa (Fluxo C)		
Módulo: Carrinho de Compras		
Gestão de Itens e Limites Operacionais		
☐		Adição e Alteração de Quantidade: Adicionar 1 produto, alterar a quantidade na interface para incrementá-lo e capturar a requisição no F12.
☐		Exclusão/Redução a Zero: Reduzir a quantidade do item para 0 ou clicar em "Remover" e capturar a requisição/resposta do servidor no F12.
☐		Validação do Servidor (F12): Alterar o payload da requisição enviando valores inválidos (-1, null, texto) e verificar a resposta do servidor.
☐		Limite de Estoque: Adicionar quantidade elevada (ex: 999 unidades) de um item e verificar se a interface/API impede a ação com base no estoque real.
Preços, Cupons e Regras Comercial - Logística		
☐		Cálculo de Frete e Prazo: Inserir um CEP válido e capturar a requisição no F12 responsável pelo retorno de prazos, transportadoras e valores.
☐		Validação de Cupom de Desconto: Inserir um cupom (válido ou sintético/inválido) e observar o recálculo via AJAX sem reflash da página.
☐		Cupons Cumulativos: Submeter múltiplos cupons em sequência para testar a regra de acúmulo de descontos da plataforma.
Recursos de IA e Personalização no Carrinho		

☐		Recomendações Dinâmicas (Cross-Selling): Inspeccionar a vitrine de “Quem comprou também levou” ou “Recomendados” e identificar a API no F12 responsável por servir esses dados.
☐		Cálculo de Frete Inteligente por Histórico: Verificar se o sistema sugere frete automaticamente baseado na localização IP/Sessão antes da digitação do CEP.
Módulo: Checkout (Identificação e Entrega)		
Sessão e Identificação		
☐		Fluxo de Identificação: Inserir um e-mail ou CPF de teste na etapa inicial para verificar se o sistema identifica se o usuário tem conta ou permite <i>Guest Checkout</i> .
☐		Preenchimento Inteligente por IA/Histórico: Testar se ao digitar e-mail/CPF, a interface sugere ou preenche automaticamente dados cadastrais prévios.
☐		Persistência de Sessão Cross-Device: Testar se o carrinho montado no desktop persiste ao logar no dispositivo móvel.
Dados de Entrega e Logística		
☐		Autocompletar de Endereço via CEP: Inserir o CEP no checkout e registrar a chamada de API que preenche automaticamente os campos de logradouro e bairro.
☐		Recálculo Dinâmico de Frete: Selecionar uma opção de entrega diferente (ex: Expressa vs. Normal) e validar a atualização dos totais do pedido.
☐		Tratamento de CEP Inexistente: Inserir um CEP sintético (ex: 00000-000) e avaliar a mensagem de validação e o estado da interface.
Módulo: Pagamento e Segurança		
Formas de Pagamento e Condicionais		
☐		Seleção de Métodos de Pagamento: Alternar entre PIX e Cartão de Crédito observando se a interface carrega novos scripts/módulos sem dar <i>refresh</i> .
☐		Validação de Cartão de Crédito: Digitar um número de cartão de teste e verificar se a validação de bandeira ocorre via script cliente (Algoritmo de Luhn).
☐		Simulação de Parcelamento: Alterar o número de parcelas no cartão de crédito e verificar se a atualização de juros/valores é calculada no cliente ou via API.
☐		Validação Dinâmica de PIX: Gerar o QR Code do PIX e verificar no F12 se há requisições repetitivas (<i>polling</i>) ou conexão <i>WebSocket</i> aguardando a confirmação do pagamento.

☐		Simulação de Recusa de Transação: Testar o comportamento da interface ao submeter um pagamento negado para verificar opções de retentativa.
Camada Antifraude e Motor de IA		
☐		Fingerprinting de Segurança: Na aba Network (F12), filtrar por termos como fraud, fingerprint, risk, clearsale ou konduto no momento do pagamento para comprovar a execução do script de análise comportamental por IA.
☐		Deteção de Comportamento Suspeito: Realizar tentativas rápidas de alteração de dados no pagamento para observar se o sistema dispara CAPTCHA ou bloqueio temporário

Tabela 1: Checklist de Inspeção e Engenharia Reversa (Fluxo C)

Legenda de Execução:



Execução Imediata (Módulo Atual): Foco na coleta de evidências para o Diagrama de Casos de Uso e Diagrama Dinâmico.



Execução Futura (Próximos Módulos): Cenários de borda avançados e resiliência de ambiente.

3. RESULTADOS OBTIDOS

Evidência 1: Seleção de Produto e Especificações (Modal/Pré-Carrinho)

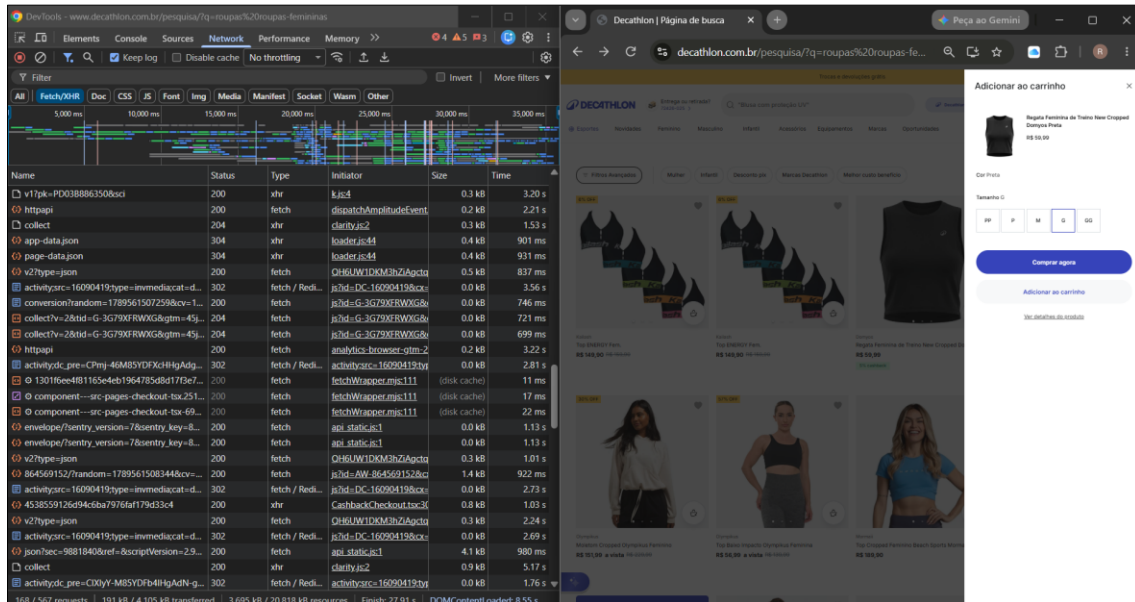


Figura 1: Seleção de Produto e Especificações

Descrição (UI):

O usuário navega pela listagem de busca e interage com um produto específico (Regata Feminina de Treino Nam Cropped Domyos Preta). Ao selecionar o produto, a interface abre um modal lateral dinâmico de pré-carrinho, com informações complementares sobre o produto (como no caso de exemplo o tamanho da regata) sem carregar a página principal, oferecendo as opções imediatas de “Comprar agora” ou “Adicionar ao carrinho”.

Análise Técnica DevTools (F12):

- `v1?pit=-PD038886350&sci...`: Chamada inicial responsável por buscar os dados da variante de produto selecionada (sku, disponibilidade do tamanho G).
- `httpapi / dispatchAmplitudeEvent`: Registro dos eventos de interação do usuário (telemetria e comportamento de clique na vitrine).
- `v2?type=json`: Atualização dos estados dinâmicos da interface (preço, promoções ativas e regras de estoque para a variação escolhida).

- Status HTTP: 200 OK (Operação de busca de dados do SKU executada com sucesso).

Evidência 2: Adição do Produto ao Carrinho e Recomendação Inteligente (IA)

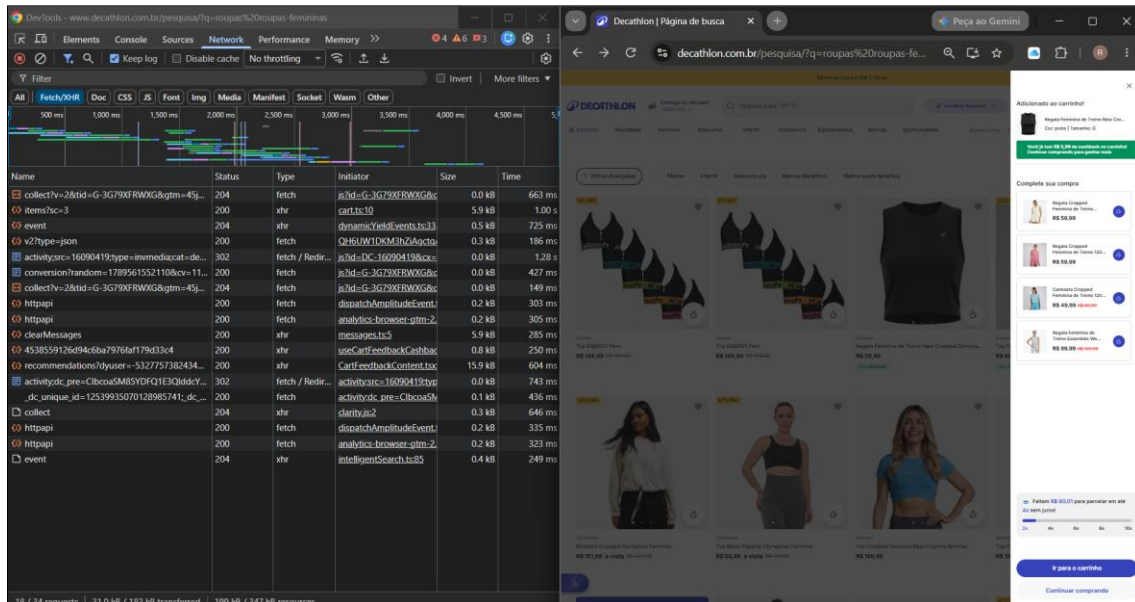


Figura 2: Adição do Produto ao Carrinho

Descrição (UI):

Ao clicar no botão “Adicionar ao carrinho” o sistema exibe a confirmação visual no modal lateral com o aviso do produto inserido. Simultaneamente o modal carrega uma vitrine dinâmica sob o título “Complete sua compra” com sugestões complementares de produtos, além de exibir a barra de progresso de benefícios (por exemplo: “Faltam R\$55,01 para parcelar”).

Análise Técnica DevTools (F12):

- items?sc=3: Requisição XHR principal responsável pela inserção do item e atualização da estrutura do carrinho (Status 200 OK, tempo de resposta: 1.00 s).
- recommendations?dyuser=-5327757382434...: Chamada disparada pelo componente CartFeedbackContent.tsx para carregar as recomendações de produtos via IA de recomendação/cross-selling (Status 200 OK, tempo de resposta: 604 ms).

- useCartFeedbackCashbac...: Chamada responsável por consultar e calcular as regras de feedback de cashback para o valor atual do carrinho (Status 200 OK)

Evidência 3: Visualização do carrinho completo; Cálculo de frete e Recomendações de página

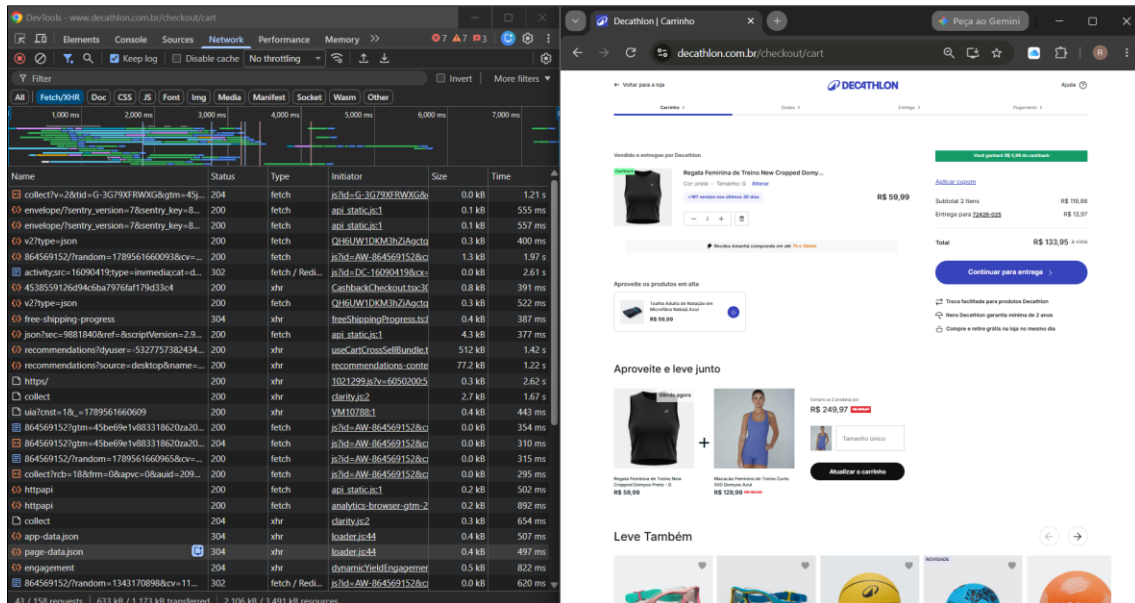


Figura 3: Visualização na página do carrinho de compras.

Descrição (UI): O usuário acessou a página dedicada do carrinho (/checkout/cart). A tela exibe o item selecionado, o seletor de quantidade, o campo de CEP preenchido automaticamente via permissão de geolocalização do navegador (sem necessidade de digitação manual pelo cliente) com valor de frete já calculado, o resumo financeiro com botão para prosseguir (“Continuar para a entrega”) e duas vitrines de recomendação personalizadas.

Análise Técnica DevTools (F12):

- free-shipping-progress: Chamada responsável por consultar e atualizar a regra de frete grátis/desconto no frete (Status 304 Not Modified).
- recommendations?source=desktop&name=...: Chamada da API de IA especificada para vitrines de página cheia (recommendations.content), retornando payload em 1.22 s (Status 200 OK).

- recommendations?dyuser=-5327757382434...: Rechamada da API Dynamic Yield (IA) para carregar o bundle de cross-selling ("Aproveite e leve junto").

Evidência 4: Alteração de Quantidade de Produtos e Recálculo Dinâmico

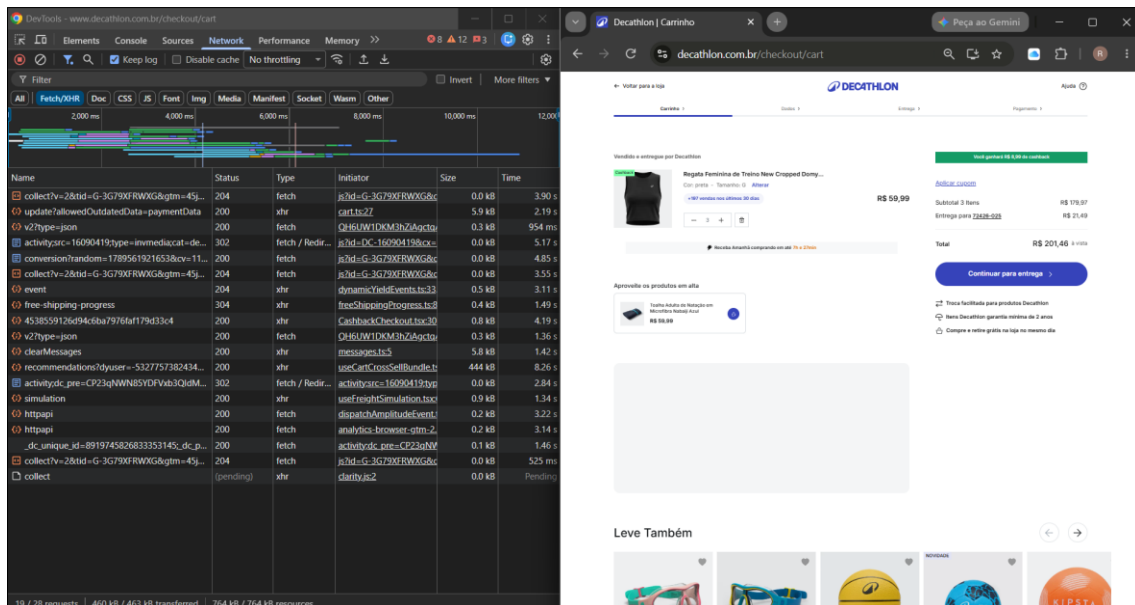


Figura 4: Alteração de quantidade de produtos no carrinho.

Descrição (UI):

O usuário alterou a quantidade do produto Regata Feminina de Treino para 3 unidades no seletor de quantidade. A interface atualizou dinamicamente o subtotal, incluindo opção de parcelamento, o frete (com base no CEP preenchido via localização) e o valor total ajustado.

Análise Técnica DevTools (F12):

- update?allowedOutdatedData=paymentData: Requisição XHR disparada pelo script cart.ts:27 para atualizar o orderForm da VTEX com a nova quantidade (Status 200 OK, tempo de resposta: 2.19 s).
- simulation: Requisição executada por useFreightSimulation.tsx para recalcular dinamicamente as tabelas de frete para a nova quantidade/peso acumulado do carrinho (Status 200 OK, 1.34 s).

- free-shipping-progress: Chamada de consulta do progresso de regras de frete grátis atualizada (Status 304 Not Modified).

Evidência 5: Remoção de Produto e Estado de Carrinho Vazio

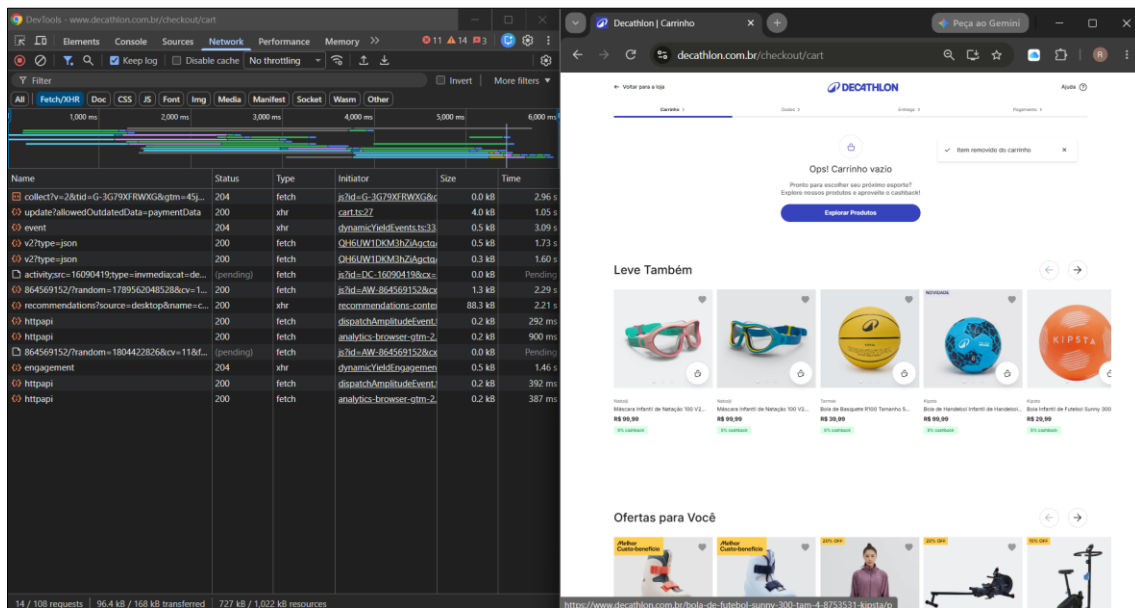


Figura 5: Remoção de produtos do carrinho.

Descrição (UI):

O usuário clicou no ícone de lixeira para remover o item do carrinho. A interface atualizou dinamicamente exibindo o aviso “Item removido do carrinho”, alterou o estado principal para a mensagem mostrada com o botão “Explorar Produtos” e renderizou novas vitrines de recomendação baseadas em outras interações do cliente.

Análise Técnica DevTools (F12):

- update?allowedOutdatedData=paymentData: Requisição XHR disparada pelo script cart.ts:27 com o payload de remoção/quantidade zero para o SKU (Status 200 OK, tempo de resposta: 1.05
- recommendations?source=desktop&name=...: Endpoint do motor de IA atualizado (Status 200 OK, 2.21 s) para reconfigurar as vitrines contextuais após a limpeza do carrinho ("Leve Também" e "Ofertas para Você")

- engagement: Registro teleférico do evento de remoção e engajamento da página (Status 204 No Content).

Evidência 6: Transição para o Checkout (Identificação / Compra rápida)

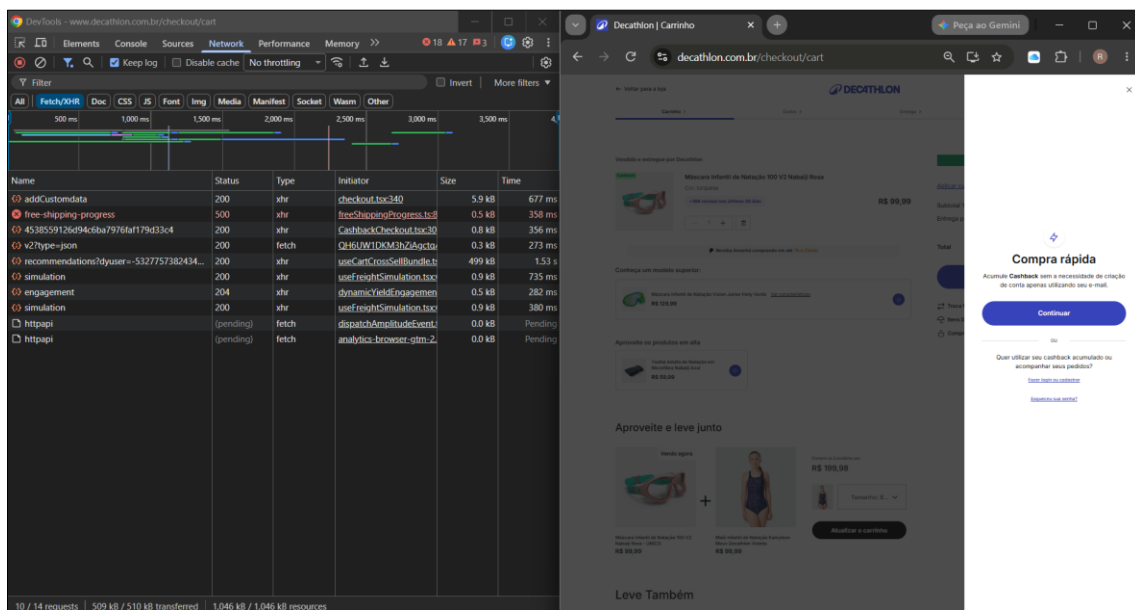


Figura 6: Página de transição para o checkout.

Descrição (UI):

Ao clicar no botão para prosseguir com a compra no carrinho, a interface abre o modal lateral “compra rápida”, incentivando a identificação simplificada via e-mail para acúmulo de cashback ou opções de login tradicional.

Análise Técnica DevTools (F12):

- addCustomData: Requisição XHR disparada pelo script checkout.tsx:340 (Status 200 OK, 677 ms) para anexar metadados da sessão ao pedido antes de avançar para a identificação.
- free-shipping-progress: Chamada que retornou erro de servidor HTTP 500 Internal Server Error em 358 ms, demonstrando um *fallback* da UI (a interface não travou e permitiu o fluxo continuar mesmo com a falha na consulta de progresso de frete).

- simulation: Duas chamadas de simulação logística processadas em paralelo com sucesso (Status 200 OK, 735 ms e 380 ms)

Evidência 7: Identificação do Usuário e Carregamento de Perfil

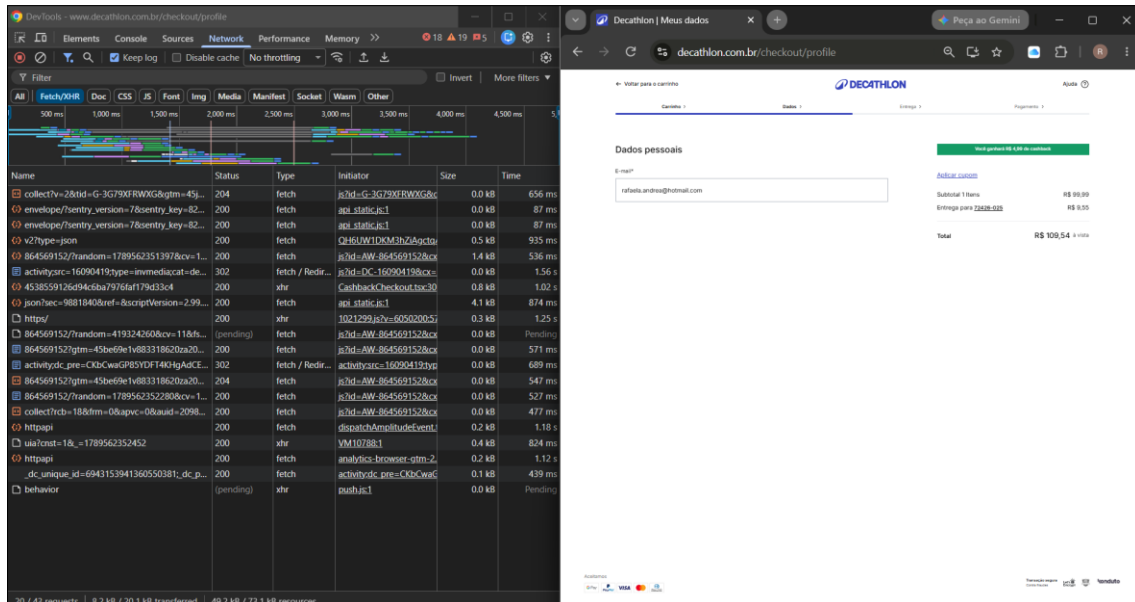


Figura 7: Identificação do usuário e Carregamento de Perfil. Fonte: Autoria própria.

Descrição (UI): O usuário avançou para a rota dedicada de identificação. A interface renderiza o formulário de “Dados Pessoais” com o e-mail preenchido, mantém o resumo financeiro atualizado à direita e exibe a faixa de benefício de cashback no topo do resumo.

Análise Técnica DevTools (F12):

- 4538559126d94c6ba7976faf179d33c4: Requisição XHR disparada por CashbackCheckout.tsx:30 (Status 200 OK, 1.02 s) para validar a elegibilidade do e-mail/sessão ao programa de cashback.
- v2?type=json: Atualização assíncrona da estrutura de estado da rota de checkout (Status 200 OK, 935 ms).
- behavior: Requisição em segundo plano (push.js:1, status pending) inicializando o monitoramento comportamental para prevenção de fraude e análise de UX.

Evidência 8: Preenchimento de Dados Cadastrais e Coleta de Comportamento (AntiFraude)

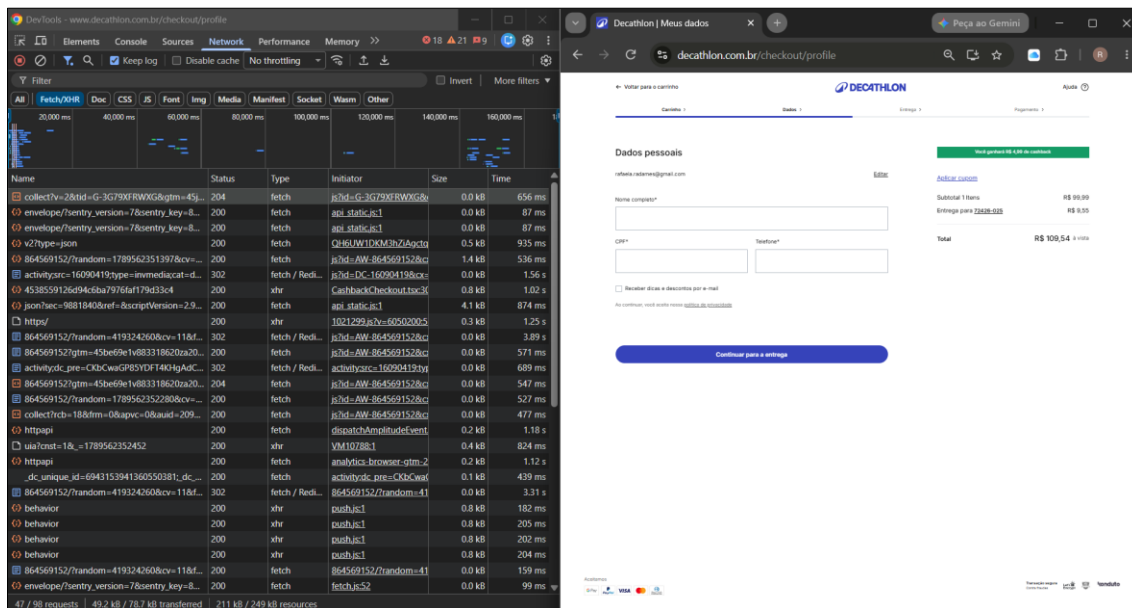


Figura 8: Preenchimento de dados cadastrais e coleta de comportamento (antifraude). Fonte: Autoria própria.

Descrição (UI): Após confirmação de e-mail, a rota expande a segunda etapa do formulário de “Dados Pessoais”. O usuário deve preencher Nome completo, CPF e Telefone para prosseguir. O botão “Continuar para a entrega” encontra-se ativo e o resumo do pedido se mantém fixo à direita.

Análise Técnica DevTools (F12):

- behavior: Sequência de 4 requisições XHR idênticas disparadas pelo script push.js:1 (Status 200 OK, tempos de resposta entre 182 ms e 205 ms). Essas chamadas coletam eventos comportamentais em tempo real (movimento do cursor, focos em campos de texto e ritmo de digitação) para alimentar o motor antifraude da plataforma (Konduto, visível no rodapé inferior direito da UI).
- envelope/?sentry_version=...: Chamadas de monitoramento de estabilidade de código e exceções em tempo real via Sentry (Status 200 OK, 99 ms).

Evidência 9: Preenchimento de Informações adicionais de Entrega e Validação de Endereço (Ressalva de UX)

Descrição (UI): O usuário acionou a etapa de entrega onde abre o painel lateral “informações adicionais”. Durante a interação ocorre o preenchimento manual dos campos Número, Complemento e Destinatário. Há um ponto de atenção de UX: a exigência rígida do campo Número obriga o preenchimento de um valor (como S/N), o que força uma simplificação do endereço que não reflete com precisão a localização exata dentro de condomínios.

Análise Técnica DevTools (F12):

- httpapi: Requisição fetch inicializada por dispatchAmplitudeEvent.js (Status 200 OK, 1.24 s), enviando os eventos de interação com os campos do formulário para o motor de analytics.
- event: Chamada fetch por intelligentSearch.ts:85 (Status 200 OK, 308 ms) para validar a consistência e busca de geolocalização do logradouro inserido.
- collect?v=2...: Sequência de requisições disparadas por js?id=G-3G79XFRWXG (Status 204 No Content / 200 OK), rastreando a conclusão do preenchimento dos campos do checkout no Google Analytics 4.

Evidência 10: Consulta Autocompletar de CEP (ViaCEP / Logística VTEX)

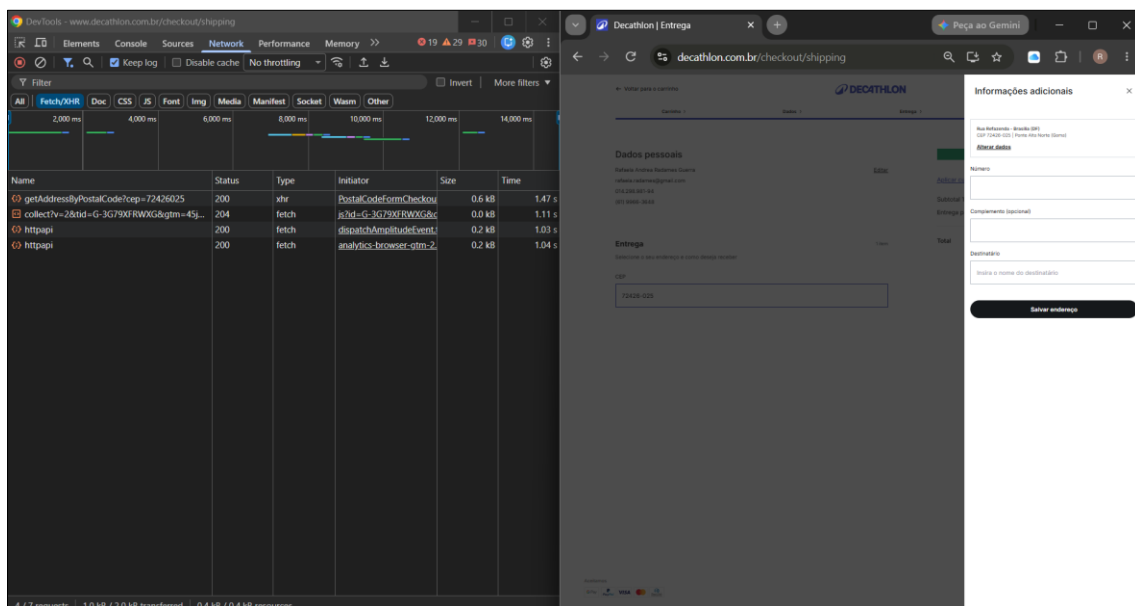


Figura 9: Consulta Autocompletar de CEP.

Descrição (UI): Após inserir o CEP na rota, a interface abre o modal lateral “Informações Adicionais” com o cabeçalho pré-preenchido automaticamente. O formulário trava os dados gerais de logradouro e solicita apenas a complementação dos campos específicos: Número, Complemento e Destinatário.

Análise Técnica DevTools (F12):

- `getAddressByPostalCode?cep=72426025`: Requisição XHR disparada pelo componente `PostalCodeFormCheckout.tsx` (Status 200 OK, tempo de resposta: 1.47 s). Essa é a chamada chave responsável por consultar o serviço de autocompletar de endereço da VTEX/ViaCEP e retornar os atributos estruturados de rua, bairro, cidade e UF.
- `httpapi`: Sequência de chamadas fetch enviando a confirmação da resolução do CEP para o motor de analytics e telemetria (Status 200 OK, 1.03 s).

Evidência 11: Seleção de Opções de Entrega e Cálculo de Frete (*shippingData*)

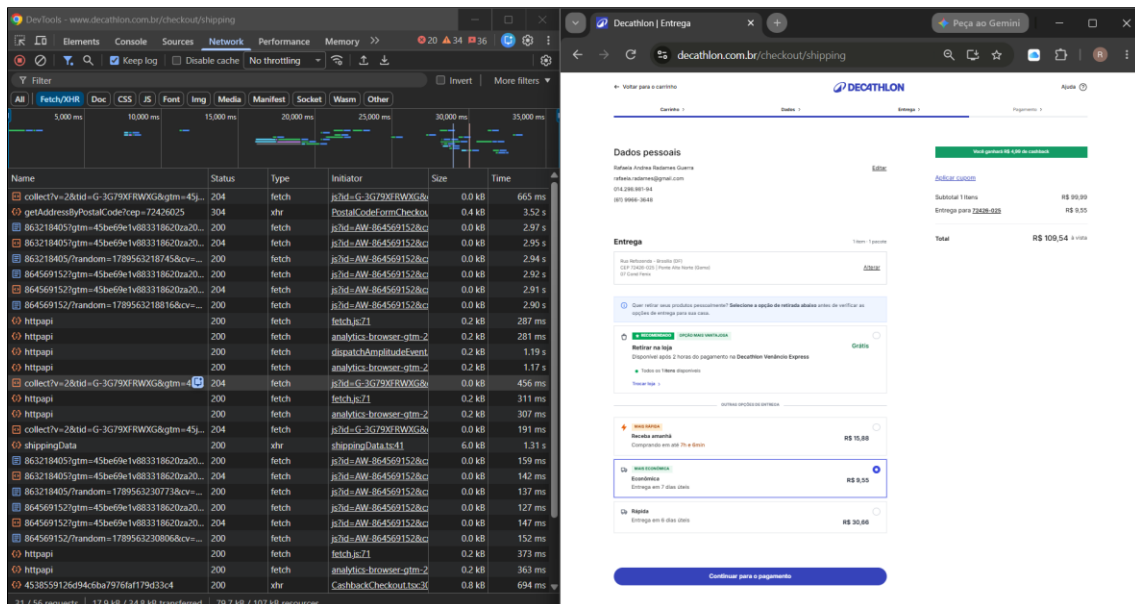


Figura 10: Seleção de opções de entrega.

Descrição (UI): Na rota `/checkout/shipping`, após a confirmação do endereço, a interface apresenta a lista de métodos de envio disponíveis:

- *Retirar na loja* (Grátis - Decathlon Venâncio Express);
- *Mais rápida* (Receba amanhã - R\$ 15,88);
- *Econômica* (Entrega em 7 dias úteis - R\$ 9,55) [Opção Seleccionada];
- *Rápida* (Entrega em 6 dias úteis - R\$ 30,66).

Ao seleccionar a opção "Econômica", o valor de R\$ 9,55 é refletido imediatamente no resumo à direita, atualizando o total do pedido para R\$ 109,54.

Análise Técnica DevTools (F12):

- shippingData: Requisição XHR disparada pelo componente shippingData.ts:41 (Status 200 OK, tempo de resposta: 1.31 s). É a chamada principal responsável por enviar o SLA de entrega escolhido ao backend da VTEX e recalculando o orderForm.
- CashbackCheckout.tsx: Requisição XHR com hash 4538559126d94c6ba7976faf179d33c4 (Status 200 OK, 694 ms), recalculando o benefício de cashback com base no valor total atualizado com o frete.
- httpapi / collect: Conjunto de chamadas disparadas por analytics-browser-gtm e dispatchAmplitudeEvent (Status 200 OK e 204 No Content) para registrar a seleção do tipo de frete nos serviços de análise.

Evidência 12: Modal de Seleção de Lojas Físicas para Retirada

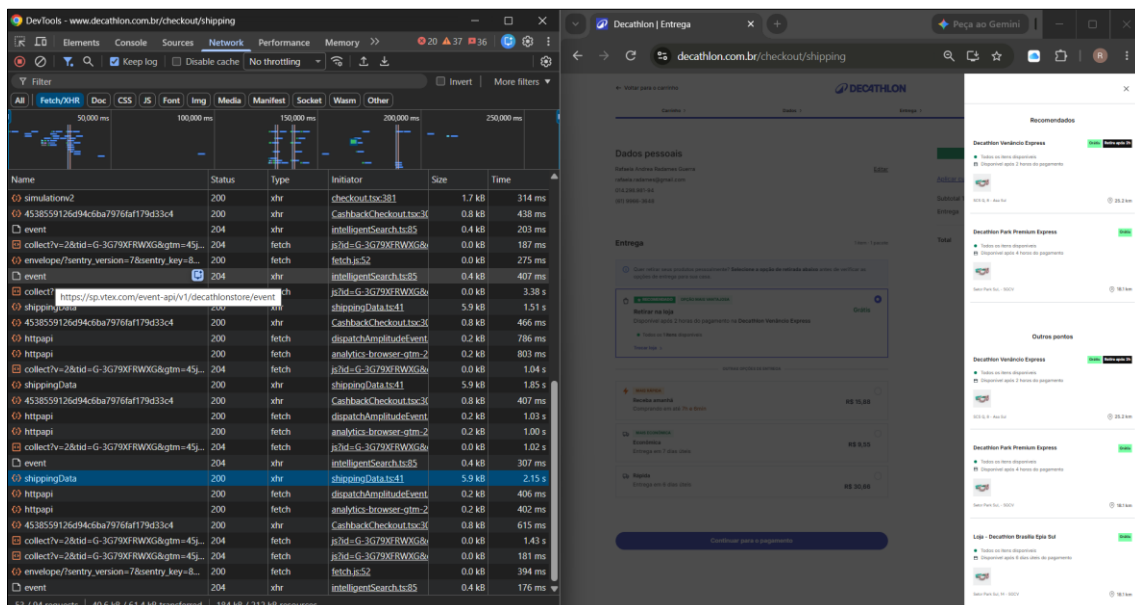


Figura 11: Modal de seleção de lojas físicas para retirada.

Descrição (UI): Na rota /checkout/shipping, ao acionar a alteração do local de retirada, abre-se um painel lateral listando os pontos físicos disponíveis com base na localização/CEP inserido:

- *Recomendados:* Decathlon Venâncio Express (Disponível após 2 horas do pagamento - Grátis);
- *Outros pontos:* Decathlon Park Premium Express (Disponível após 6 horas do pagamento - Grátis) e Loja - Decathlon Brasília Epia Sul.

Análise Técnica DevTools (F12):

- simulationv2: Requisição XHR iniciada por checkout.tsx:381 (Status 200 OK, tempo de resposta: 314 ms). É a chamada responsável por consultar o inventário das lojas físicas e calcular o SLA de prontidão (retirada em 2h ou 6h).
- shippingData: Chamada XHR disparada por shippingData.ts:41 (Status 200 OK, 2.15 s), atualizando o orderForm da VTEX com a loja física escolhida como ponto de entrega (pickup-point).
- event: Requisições XHR para intelligentSearch.ts:85 (Status 200 OK, 203 ms / 176 ms) e chamadas sp.vtex.com/event-api/v1/decathlonstore/event, rastreando a interação com os pontos de retirada e disponibilidade por geolocalização.

Evidência 13: Seleção do Método do Pagamento e Inicialização do Checkout Final

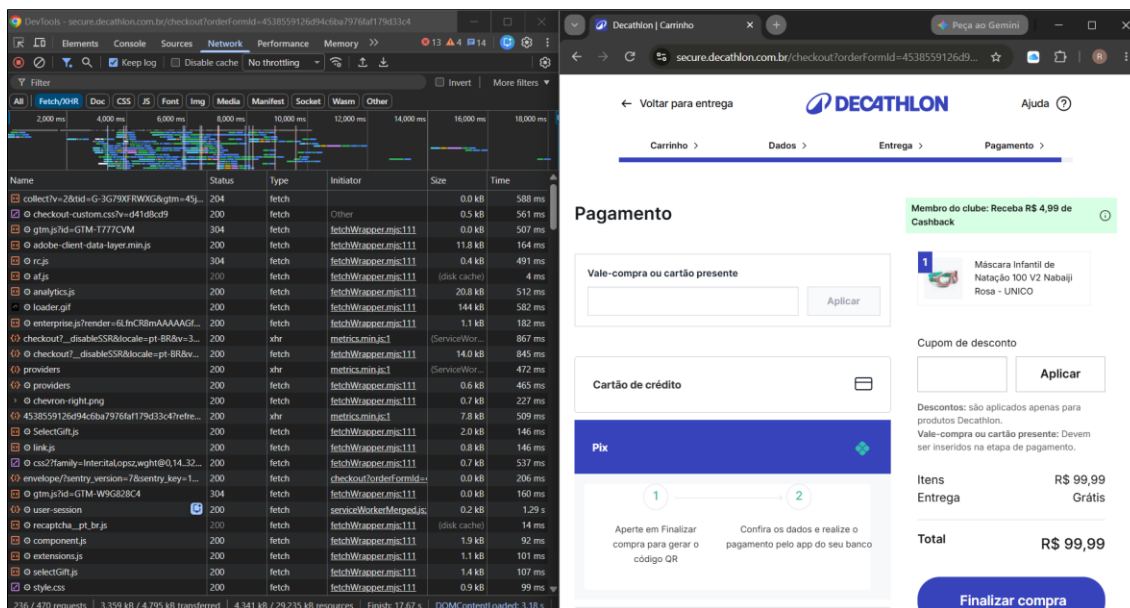


Figura 12: Seleção de método de pagamento.

Descrição (UI): Na etapa final `/checkout?orderFormId=4538559126d94c6ba7976faf179d33c4`, o cliente acessa a aba "Pagamento". A interface expõe os métodos aceitos:

- *Vale-compra ou cartão presente* (com campo de aplicação dedicado);
 - *Cartão de crédito*;
 - **Pix [Opção Selecionada]** (apresentando a instrução passo a passo para geração do QR Code após o clique em "Finalizar compra").
- No painel lateral direito, são exibidos o cupom de desconto, o resumo do produto (*Máscara Infantil de Natação 100 V2 Nabaiji*), o benefício de cashback (*R\$ 4,99*) e o valor total final (**R\$ 99,99** com frete grátis).

Análise Técnica DevTools (F12):

- `4538559126d94c6ba7976faf179d33c4?refre...`: Requisição XHR disparada por `metrics.min.js:1` (Status 200 OK, 509 ms), atualizando a instância do `orderForm` no contexto de pagamento.
- `recaptcha_pt_br.js / enterprise.js?render=...`: Requisição enviada via `fetchWrapper.mjs:111` (Status 200 OK, 182 ms), inicializando o serviço Google reCAPTCHA Enterprise para validação antifraude antes da emissão da ordem de pagamento.

- user-session: Requisição fetch iniciada por serviceWorkerMerged.js (Status 200 OK, 1.29 s), autenticando a sessão ativa do usuário para associar os dados de pagamento de forma segura.
- providers: Chamadas XHR e fetch para obtenção dos provedores de pagamento homologados na VTEX Smart Checkout (Status 200 OK, 465 ms / 472 ms).

Evidência 14: Seleção do Método de Pagamento via Cartão de Crédito e Telemetria Antifraude (paymentData)

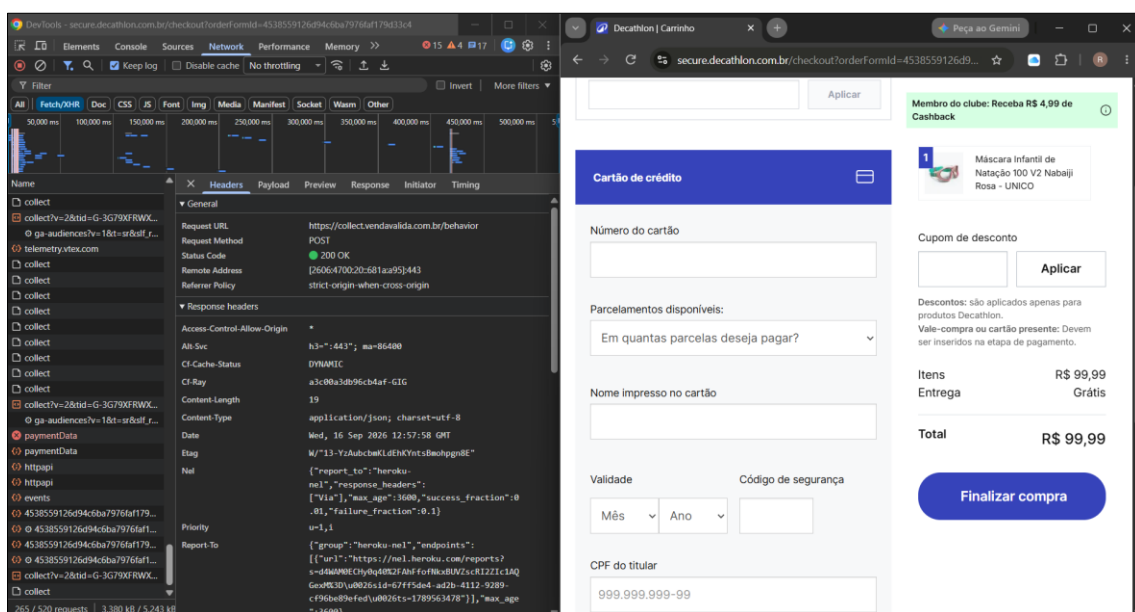


Figura 13: Seleção do método de pagamento: cartão de crédito. Fonte: Autoria própria.

Descrição (UD): Na rota de pagamento (/checkout?orderFormId=4538559126d94c6ba7976faf179d33c4), ao selecionar a opção Cartão de Crédito, a interface expande dinamicamente o formulário específico contendo os campos:

- *Número do cartão;*
- *Parcelamentos disponíveis* (menu drop-down "Em quantas parcelas deseja pagar?");
- *Nome impresso no cartão;*
- *Validade* (Mês / Ano);
- *Código de segurança* (CVV);

- *CPF do titular* (pré-formatado com a máscara 999.999.999-99).

Análise Técnica DevTools (F12):

- https://collect.vendavalida.com.br/behavior: Requisição POST efetuada via Fetch/XHR (Status 200 OK), responsável por coletar dados comportamentais do usuário (biometria comportamental) para avaliação antifraude antes do envio dos dados financeiros.
- *paymentData*: Requisição XHR disparada (Status 200 OK), atualizando a seção de pagamentos do orderForm no contexto VTEX para registrar a alteração do método selecionado de Pix para Cartão de Crédito.
- *telemetry.vtex.com*: Comunicação fetch ativa para envio de diagnósticos de performance do checkout.

Evidência 15: Seleção e Expansão do Formulário de Cartão de Crédito

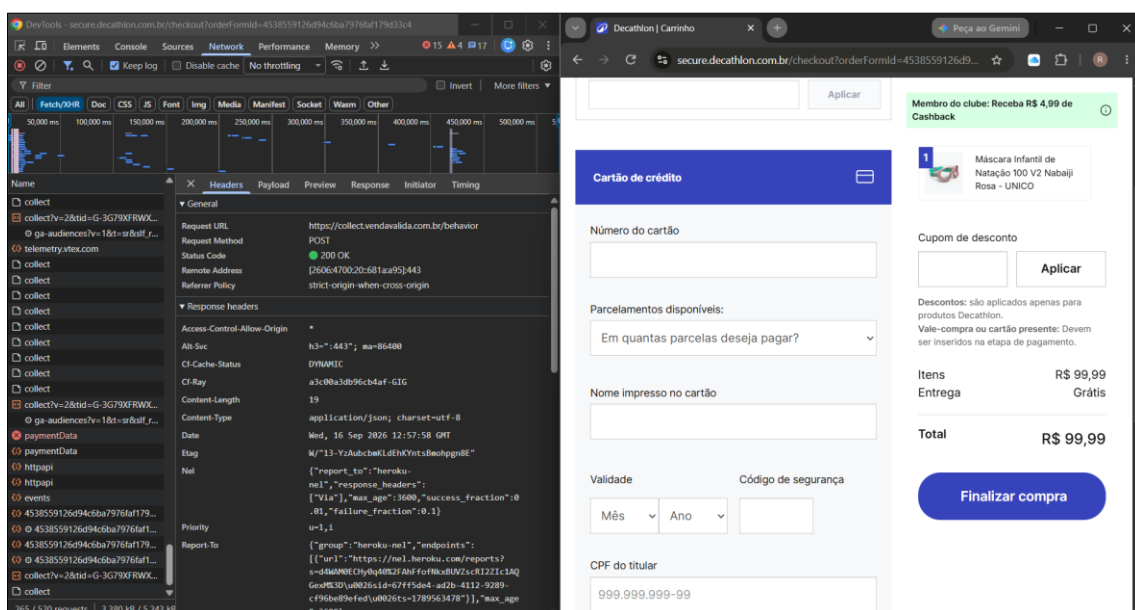


Figura 14: Expansão do formulário de pagamento com cartão de crédito, integração antifraude e atualização do estado do checkout. Fonte: Autoria própria.

Descrição (UI): Na etapa final de pagamento (/checkout?orderFormId=4538559126d94c6ba7976faf179d33c4), ao selecionar o método Cartão de crédito, a UI expande a seção contendo os dados de entrada:

- *Número do cartão;*
- *Parcelamentos disponíveis* (menu suspenso com seleção de parcelas);
- *Nome impresso no cartão;*
- *Validade* (Mês / Ano);
- *Código de segurança* (CVV);
- *CPF do titular* (campo com máscara preenchida).

No painel à direita, mantém-se a exibição do resumo do pedido contendo a *Máscara Infantil de Natação 100 V2 Nabaiji* (R\$ 99,99), cashback acumulado (R\$ 4,99) e frete grátis.

Análise Técnica DevTools (F12):

- https://collect.vendavalida.com.br/behavior: Requisição POST efetuada (Status 200 OK), responsável por coletar dados de comportamento da sessão (fingerprint/biometria comportamental) para avaliação do motor antifraude.
- paymentData: Requisições XHR efetuadas (Status 200 OK), registrando a alteração no método de pagamento ativo no orderForm da VTEX.
- telemetry.vtex.com: Chamada de diagnóstico de performance e tempo de resposta da interface de checkout.

Evidência 16: Consulta do BIN do Cartão e Seleção de Parcelamento

The screenshot displays the checkout interface of a Decathlon store. On the left, the Chrome DevTools Network tab is open, showing a list of network requests. The 'paymentData' request is highlighted, indicating a successful POST request (Status 200 OK). The right side of the image shows the checkout page with a credit card form. The card number is 4641 2799 8247 3313, and the expiration date is 05/28. The cardholder's name is Ráfaela Andréa. The page also shows a coupon code field and a 'Finalizar compra' button. The total amount is R\$ 99,99, and the cashback is R\$ 4,99.

Figura 15: Consulta dinâmica de parcelamento por BIN do cartão de crédito e sincronização do orderForm. Fonte: Autoria própria.

Descrição (UI): Na rota de pagamento (/checkout?orderFormId=4538559126d94c6ba7976faf179d33c4), o usuário preenche o número do cartão (4641 2799 8247 3313), o que ativa o reconhecimento da bandeira VISA. A partir desse preenchimento, o campo suspenso "Parcelamentos disponíveis" exibe dinamicamente as plano de pagamento aceitos:

- *Pagamento à vista - R\$ 99,99;*
- *2x de R\$ 49,99 sem juros [Opção Selecionada]. Também são informados os dados de Validade (05/28), Nome impresso (Rafaela Andrea) e CPF do titular.*

Análise Técnica DevTools (F12):

- **paymentData:** Requisições XHR disparadas por metrics.min.js:1 (Status 200 OK, 2.15 s e 1.21 s). Nota-se o cancelamento prévio (canceled) de chamadas intermediárias enquanto o usuário digitava, otimizando o envio final com os dados consolidados do parcelamento escolhido.
- **4538559126d94c6ba7976faf179d33c4:** Múltiplas chamadas fetch e xhr via FetchWrapper.mjs:111 e metrics.min.js:1 (Status 200 OK, variando entre 520 ms e 2.94 s), revalidando a árvore do orderForm no backend VTEX com a regra de parcelamento aplicada.
- **events / httpapi:** Requisições xhr e fetch (Status 200 OK, 373 ms a 1.34 s), registrando eventos de input e seleção no analytics.

Evidência 17: Alternância para Pagamento via PIX e Sincronização de Estado

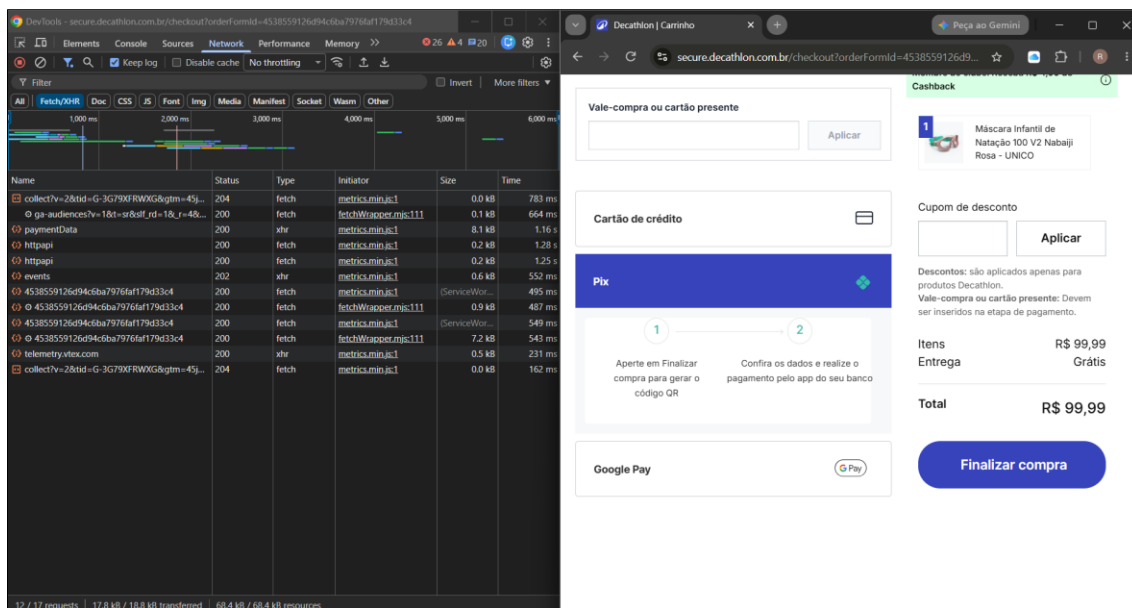


Figura 16: Seleção da modalidade PIX, atualização assíncrona do contexto de pagamento e prontidão para fechamento do pedido. Fonte: Autoria própria.

Descrição (UI): Na rota `/checkout?orderFormId=4538559126d94c6ba7976faf179d33c4`, o cliente altera a opção de pagamento de Cartão de Crédito para Pix. A UI recolhe o formulário de cartão e exibe o componente contendo os passos instrutivos: 1. *Aperte em Finalizar compra para gerar o código QR* e 2. *Confira os dados e realize o pagamento pelo app do seu banco*. Abaixo, o sistema disponibiliza também a opção Express de pagamento via Google Pay.

Análise Técnica DevTools (F12):

- `paymentData`: Requisição XHR efetuada por `metrics.min.js:1` (Status 200 OK, tempo de resposta: 1.16 s, tamanho: 8.1 kB). Essa chamada é o elemento chave que notifica o backend VTEX da mudança da opção de pagamento para Pix, ajustando as regras de liquidação do pedido.
- `4538559126d94c6ba7976faf179d33c4`: Múltiplas requisições `fetch` via `fetchWrapper.mjs:111` e `ServiceWorker` (Status 200 OK, tempos entre 487 ms e 549 ms), sincronizando a árvore do `orderForm` atualizada.
- `events` / `httpapi`: Requisições `xhr` e `fetch` (Status 202 Accepted / 200 OK, 552 ms a 1.28 s), registrando no analytics a transição da modalidade de pagamento.

Evidência 18: Validação de Regra de Negócio – Tentativa de Aplicação de Cupom Inválido

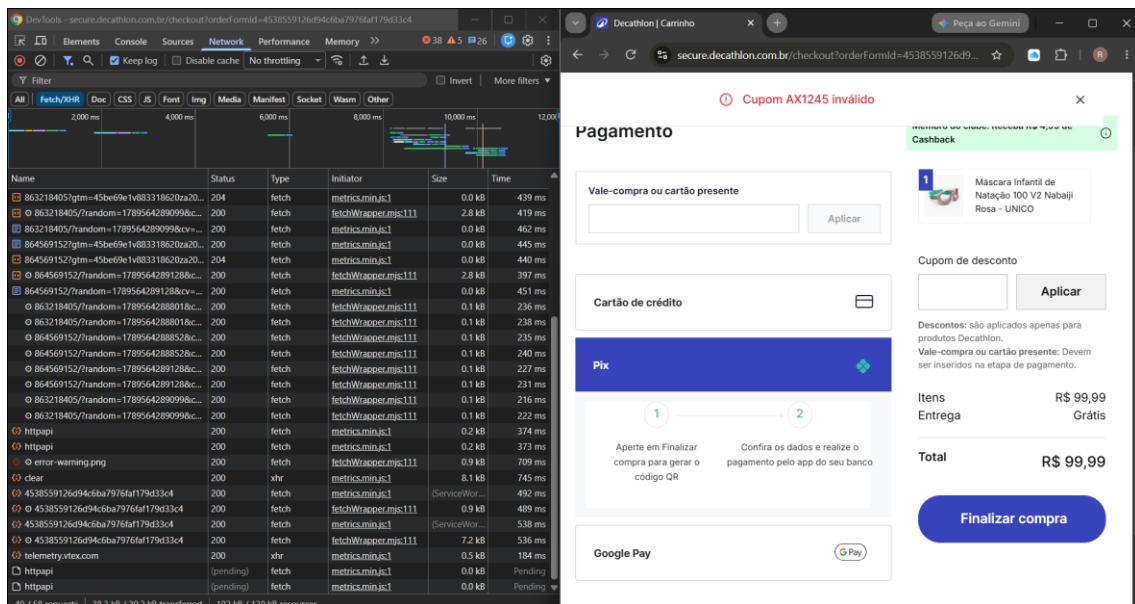


Figura 17: Validação de regra de negócio, tratamento de exceção em cupom promocional inválido e execução da limpeza de contexto. Fonte: Autoria própria.

Descrição (UI): Na etapa de pagamento do checkout (/checkout?orderFormId=4538559126d94c6ba7976faf179d33c4), o usuário inseriu o código AX1245 no campo "Cupom de desconto" e acionou o botão "Aplicar". A interface tratou a resposta do backend exibindo um toast/banner de alerta no topo superior direito com a mensagem Cupom AX1245 inválido, acompanhado do ícone error-warning.png. O valor total do pedido permaneceu inalterado em R\$ 99,99.

Análise Técnica DevTools (F12):

- clear: Requisição XHR disparada por metrics.min.js:1 (Status 200 OK, tempo de resposta: 745 ms, tamanho: 8.1 kB). Essa chamada é responsável por redefinir e remover a tentativa de aplicação do voucher inválido da estrutura de cupons do checkout.
- error-warning.png: Requisição fetch via fetchWrapper.mjs:111 (Status 200 OK, 709 ms), realizando a renderização do ativo visual do alerta de erro na UI.

- 4538559126d94c6ba7976faf179d33c4: Múltiplas chamadas fetch via ServiceWorker (Status 200 OK, tempos entre 489 ms e 538 ms), garantindo a revalidação da árvore do orderForm sem nenhum desconto concedido.
- httpapi / telemetry.vtex.com: Disparo de chamadas de análise (Status 200 OK, 184 ms a 374 ms), registrando o evento de falha na aplicação de cupom promocional para métricas de conversão.

4. ANÁLISE CRÍTICA DOS RESULTADOS

A análise dos resultados obtidos através da inspeção de tráfego de rede via Google Chrome DevTools (F12) permitiu convalidar a execução dos requisitos funcionais, o comportamento dos serviços assíncronos e a resiliência da arquitetura orientada a microsserviços da plataforma VTEX. A seguir, detalham-se as evidências técnicas capturadas ao longo de toda a jornada do usuário no e-commerce.

4.1. Navegação, Seleção de Produtos e Motor de Recomendação

Na etapa inicial de navegação e seleção de produto (Figura 1), a inspeção evidenciou a reatividade do front-end ao selecionar variações de mercadoria (como o tamanho G na Regata Feminina de Treino Nam Cropped). A ação disparou requisições assíncronas via Fetch/XHR aos endpoints `v1?pit` e `v2?type=json` (ambos com status HTTP 200 OK), consultando os serviços de catálogo e estoque da VTEX. Essas chamadas permitiram renderizar o modal de pré-carrinho sem requerer o recarregamento completo da página (full page refresh), provendo as premissas para a inicialização do fluxo de compras e cancelando o UC01 (Selecionar Produto).

Ao adicionar o item ao carrinho (Figura 2), confirmou-se o processamento assíncrono das regras de negócio pela requisição `items?sc=3` (HTTP 200 OK), encarregada de persistir o produto na sessão ativa. Concomitantemente, evidenciou-se a atuação do motor de Inteligência Artificial da plataforma por meio do endpoint `recommendations?dyuser=...` (HTTP 200 OK, tempo de resposta de 604 ms), responsável por computar e renderizar em tempo real as sugestões de cross-selling da seção "Complete sua compra".

4.2. Gestão do Carrinho de Compras, Logística e Reatividade de Estado

A transição para a página dedicada do carrinho (`/checkout/cart`, Figura 3) consolidou a gestão de itens, cálculo de frete e recomendações dinâmicas. Verificaram-se as chamadas bem-sucedidas aos endpoints `free-shipping-progress` e `recommendations?source=desktop` (HTTP 200 / 304). O motor de IA (`recommendations.content`, 1,22 s) alimentou as vitrines "Aproveite e leve junto" e "Leve

Também", enquanto o serviço logístico retornou a precificação de entrega (R\$ 13,01) para o CEP informado, embasando o UC04 (Visualizar Carrinho) e o UC05 (Calcular Frete).

Ao alterar a quantidade do item para 3 unidades (Figura 4), a plataforma demonstrou alta reatividade. O back-end executou a chamada `update?allowedOutdatedData=paymentData` (HTTP 200 OK) para atualizar a estrutura de dados do pedido (`orderForm`). Sequencialmente, a alteração no volume e peso das mercadorias disparou o endpoint `simulation` (HTTP 200 OK), reajustando assincronamente o valor do frete para R\$ 21,49 e o total do carrinho para R\$ 201,46, o que homologa o UC06 (Alterar Quantidade) e revalida o UC05 (Calcular Frete).

No cenário oposto, ao realizar a remoção completa dos itens (Figura 5), o acionamento da requisição `update?allowedOutdatedData=paymentData` (HTTP 200 OK) zerou os dados do `orderForm`. O front-end atualizou a interface para a mensagem "Ops! Carrinho vazio" e disparou a chamada `recommendations?source=desktop` (HTTP 200 OK), reconfigurando as vitrines para o modo de retenção e descoberta ("Ofertas para Você"), o que valida o UC07 (Remover Item do Carrinho).

4.3. Transição para o Checkout, Identificação e Segurança Antifraude

O avanço para a etapa de checkout ("Compra rápida", Figura 6) acionou a camada de identificação dinâmica. Destaca-se a requisição `addCustomData` (HTTP 200 OK, 677 ms), responsável por vincular parâmetros da sessão atual ao `orderForm`. Durante esta transição, identificou-se uma exceção de back-end no endpoint `free-shipping-progress` (HTTP 500 Internal Server Error); contudo, a arquitetura do front-end tratou a falha de forma resiliente, exibindo o modal de autenticação sem interromper a navegação, o que sustenta o UC08 (Iniciar Checkout).

A entrada na rota `/checkout/profile` (Figura 7) deu início ao Módulo de Checkout. A requisição acionada por `CashbackCheckout.tsx` (HTTP 200 OK, 1,02 s) calculou a rentabilidade do cashback (R\$ 4,99) a partir do e-mail informado. Em paralelo, o endpoint `v2?type=json` (HTTP 200 OK) garantiu a consistência dos dados do pedido no lado do cliente, fundamentando o UC10 (Identificar Cliente) e o UC11 (Consultar Cashback).

Durante a expansão dos dados cadastrais (Nome, CPF e Telefone), observou-se a atuação da camada de segurança contra fraudes. O script `push.js` disparou requisições em rajada ao endpoint `behavior` (todas com HTTP 200 OK, com tempo médio de ~200 ms), transmitindo a telemetria do preenchimento para processamento junto ao motor antifraude

integrado (Konduto). Essa evidência chancela o UC12 (Preencher Dados Cadastrais) e o UC13 (Coletar Telemetria Antifraude).

4.4. Logística de Entrega, Autocompletar e Retirada em Loja Física (Pickup Points)

Na rota de entrega (/checkout/shipping), a consulta de CEP para preenchimento de endereço ativou o componente PostalCodeFormCheckout.tsx, disparando o endpoint getAddressByPostalCode?cep=72426025 (HTTP 200 OK, 1,47 s). A resposta preencheu automaticamente os campos de logradouro, bairro, cidade e estado, cabendo ao usuário apenas a complementação do número e destinatário, o que valida o UC14 (Inserir Endereço de Entrega) e o UC15 (Consultar Endereço por CEP).

Em seguida, a definição do frete acionou a chamada shippingData (HTTP 200 OK, 1,31 s, originada em shippingData.ts:41). A seleção do SLA "Econômica" atualizou o custo de entrega para R\$ 9,55 e o total do pedido para R\$ 109,54. Concomitantemente, o componente CashbackCheckout.tsx (HTTP 200 OK, 694 ms) recalculou o crédito de cashback proporcional, homologando o UC16 (Selecionar Modalidade de Entrega) e o UC04 (Visualizar Resumo do Pedido).

Alternativamente, a seleção da modalidade de retirada física abriu o painel lateral de pontos de coleta (Pickup Points). A requisição simulationv2 (HTTP 200 OK, 314 ms, disparada por checkout.tsx:381) executou a simulação de estoque por filial em tempo real, calculando os prazos de disponibilidade (2 horas na loja Venâncio Express e 6 horas na Park Premium Express). A interação atualizou o orderForm via chamadas a shippingData (HTTP 200 OK, 2,15 s) e buscas em intelligentSearch.ts:85, convalidando o UC17 (Selecionar Ponto de Retirada em Loja Física).

4.5. Processamento de Pagamento, Antifraude e Validação de Regras de Negócio

A rota final de pagamento (/checkout?orderFormId=4538...) envolveu a obtenção das regras de aquisição via endpoint providers (HTTP 200 OK, 472 ms) e a sincronização contínua da instância do pedido (HTTP 200 OK, 509 ms). Para assegurar a integridade financeira, o sistema carregou de forma assíncrona o script recaptcha_pt_br.js (HTTP 200 OK, 182 ms) e validou a sessão do usuário em user-session (HTTP 200 OK,

1,29 s), sustentando a camada de segurança do UC04 e o UC18 (Selecionar Forma de Pagamento).

A escolha do Cartão de Crédito expandiu o formulário de captura e enviou os dados atualizados ao backend via endpoint `paymentData` (HTTP 200 OK). Paralelamente, registrou-se o envio de requisição POST para `collect.vendavalida.com.br/behavior` (HTTP 200 OK), acionando mecanismos de biometria comportamental e fingerprint digital para mitigação de risco de fraude. Essa etapa valida os requisitos do UC21 (Preencher Dados do Cartão de Crédito).

Com a digitação do número do cartão, a identificação automática do BIN (bandeira Visa) enviou payloads ao endpoint `paymentData` (HTTP 200 OK, com tempos de 2,15 s e 1,21 s, disparados via `metrics.min.js:1`). O sistema habilitou o menu de parcelamento (opção "2x de R\$ 49,99 sem juros") e consolidou a estrutura de pagamento na chamada final do `orderFormId` (HTTP 200 OK), homologando o UC20 (Selecionar Opções de Parcelamento) e revalidando o UC21.

A alteração para a modalidade Pix demonstrou a flexibilidade do checkout. O endpoint `paymentData` (HTTP 200 OK, 1,16 s, 8,1 kB) atualizou a preferência financeira no `orderFormId`, reestruturando a interface para a futura emissão do QR Code e Pix Copia e Cola, além de expor a carteira digital Google Pay como opção secundária, o que chancela o UC18 (Selecionar Forma de Pagamento).

Por fim, o teste de submissão de um cupom promocional inexistente (AX1245) comprovou a robustez do tratamento de exceções e regras de negócio. O back-end executou a chamada ao endpoint `clear` (HTTP 200 OK, 745 ms, 8,1 kB), expurgando a tentativa de desconto e atualizando a instância do `orderFormId` (HTTP 200 OK). O front-end tratou o retorno ao carregar o ícone `error-warning.png` (HTTP 200 OK, 709 ms) e apresentar o alerta visual "Cupom AX1245 inválido", mantendo a integridade dos valores financeiros recalculados e convalidando o fluxo de exceção do UC22 (Aplicar Cupom de Desconto).

5. CONSIDERAÇÕES FINAIS

A condução deste trabalho permitiu mapear, modelar e convalidar detalhadamente a arquitetura de software e a dinâmica de funcionamento de um e-commerce de grande porte respaldado pela plataforma VTEX. A combinação de engenharia reversa — viabilizada pela inspeção contínua de requisições de rede via Google Chrome DevTools — com os preceitos formais da Notação UML (Diagramas de Casos de Uso, Classes, Pacotes e Sequência) garantiu uma visão holística e tecnicamente fundamentada do sistema.

A análise empírica dos resultados evidenciou que a plataforma adota uma arquitetura altamente reativa, resiliente e orientada a microsserviços. Observou-se a descentralização de responsabilidades em serviços especializados de catálogo, logística, motor de recomendação baseado em Inteligência Artificial, gateways de pagamento e camadas de segurança comportamental antifraude (Konduto/Vendavalida). A persistência e sincronização contínua do estado do pedido por meio do objeto `orderForm` garantiu a integridade das transações mesmo diante de exceções de rede e tratamentos de erros no front-end, comprovando a robustez do ecossistema.

Do ponto de vista metodológico e de engenharia de equipe, o desenvolvimento dos artefatos priorizou a co-criação, a divisão modular da documentação e a revisão estrita em pares (Peer Review). A tomada de decisões arquiteturais e o alinhamento tático foram impulsionados por dinâmicas ágeis de comunicação assíncrona e debates em sala de aula invertida. Complementarmente, o uso do senso crítico sobre ferramentas de IA Generativa atuou como um catalisador no refinamento das análises e na consolidação da documentação técnica.

Em suma, o projeto cumpriu integralmente seus objetivos ao preencher a lacuna entre a teoria da modelagem de software e a prática da engenharia de sistemas em nível industrial, consolidando aprendizados valiosos sobre rastreabilidade, padrões de projeto e arquitetura de aplicações web de alta complexidade.

REFERENCIAS BIBLIOGRÁFICAS

MYERS, Glenford J.; SANDLER, Corey; BADGETT, Tom. The Art of Software Testing. 3. ed. Hoboken: John Wiley & Sons, 2012.

CANHOTA JUNIOR, Antonio Jorge Sapage da; SOUZA, Diego Alves de; MOUTINHO, Diogo dos Santos; LOHNEFINK, Felipe Paixão. Engenharia Reversa. 2005. Trabalho apresentado à disciplina Informática I, Curso de Ciência da Computação, Universidade Federal Fluminense, Niterói, 2005.

VALENTE, Marco Tulio. Engenharia de Software Moderna: Princípios e Práticas para Desenvolvimento de Software com Produtividade. Disponível em: <https://engsoftmoderna.info/>